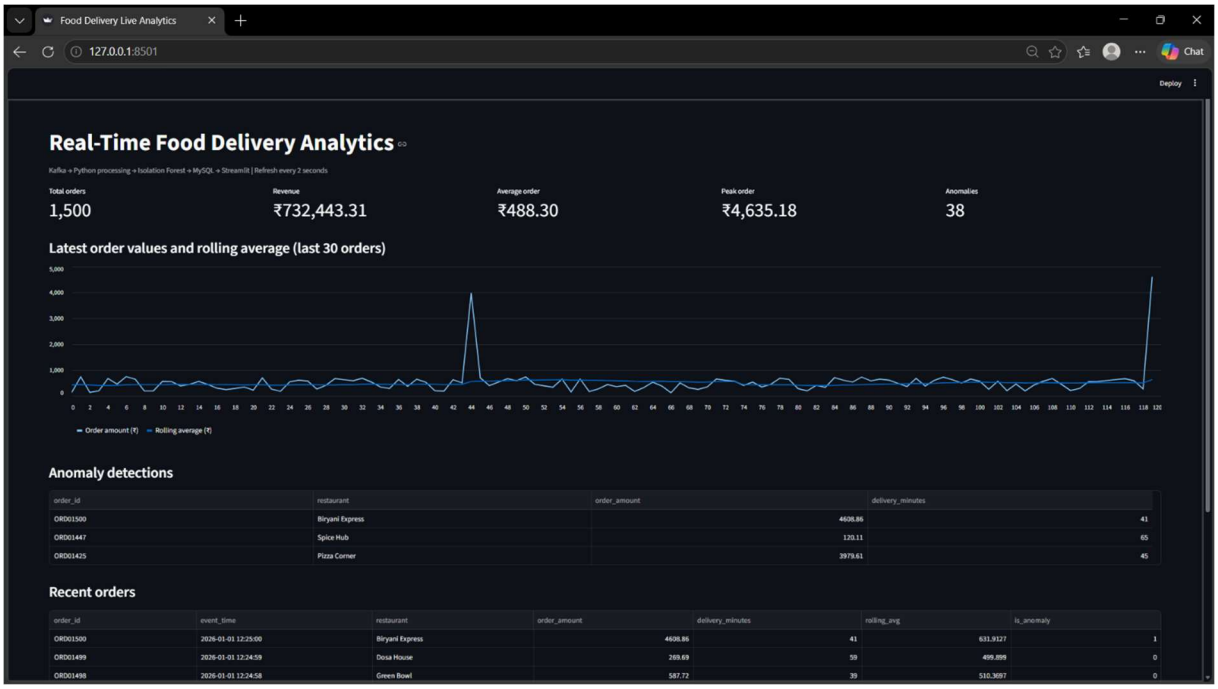
REAL-TIME FOOD DELIVERY ANALYTICS AND ANOMALY DETECTION

Mini-Project Report Aligned to "Real-Time Streaming Analytics: A Student Guide to Building a Mini-Project"

Submitted by: DHARUN P
USN: 23BTRCL014

Course: Data Stream Analytics



Cover evidence: working food-delivery analytics dashboard.

1. Abstract

A six-layer streaming analytics mini-project was built using synthetic food-delivery order data. Python publishes JSON events to Apache Kafka; a consumer calculates rolling statistics and uses an offline-trained Isolation Forest to flag unusual order values or delivery durations. MySQL stores individual events and running totals, while Streamlit refreshes the dashboard every two seconds. An initial 1,500-order run was followed by a verified 20-order live demonstration. Final results were 1,520 orders, ₹748,233.31 in revenue and 40 flagged anomalies, including two detected during the live demonstration.

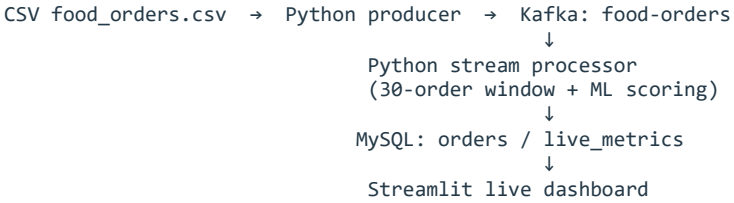
2. Problem Statement and Objectives

A delivery platform needs to monitor changing order activity and unusual transactions as events arrive. Batch-only summaries cannot demonstrate the live progression of order counts and trends. This project models a streaming alternative using simulated orders, not real customer data.

- Transport events using Kafka.
- Compute the last-30-orders rolling average and running aggregates.
- Train Isolation Forest offline and score incoming events.
- Store history and current metrics in MySQL.
- Show auto-refreshing metrics, trends and anomaly records in Streamlit.

3. Six-Layer System Architecture

Layer	Selected tool	Responsibility
1. Data source	Synthetic CSV	Generate 1,500 baseline orders and 20 unique live-demo events
2. Streaming	Apache Kafka	Transport JSON events via food-orders
3. Processing	Python consumer	Windowed average and running totals
4. Machine learning	Isolation Forest	Score each order for unusual behavior
5. Serving database	MySQL	Persist history and summary metrics
6. Visualization	Streamlit	Read current data and refresh every 2 s



Kafka decouples the producer from the processor; MySQL decouples processing from visualization. The dashboard is live only when fresh events are processed and the browser re-queries automatically.

4. Dataset (Layer 1)

The initial generated dataset contains 1,500 synthetic orders. Each event has order_id, event_time, restaurant, order_amount and delivery_minutes. For the separate live demonstration, demo_producer.py published 20 additional synthetic JSON orders with new IDs ORD01501–ORD01520, preserving the original dataset and database rows.

```
{"order_id":"ORD00001","event_time":"2026-01-01T12:00:01",
 "restaurant":"Spice Hub","order_amount":522.84,"delivery_minutes":62}
```

5. Kafka Producer and Topic (Layer 2)

The Python producer reads the CSV, converts fields to numeric types, sends each JSON event to food-orders, waits for broker acknowledgement and sleeps between messages to simulate a live feed. The verified topic uses one partition and replication factor one.

```
producer = KafkaProducer(bootstrap_servers=BROKER,
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    acks='all', retries=5)
producer.send(TOPIC, value=event).get(timeout=15)
time.sleep(max(0, args.delay))
```

```
E:\DSA MINI PROJECT\Kafka_Food_Delivery_Mini_Project\KafkaFoodDeliveryMiniProject>cd /d C:\kafka

C:\kafka>bin\windows\kafka-topics.bat --bootstrap-server localhost:9092 --describe --topic food-orders
Topic: food-orders      TopicId: Ps-tZbDYQAybxVffcKwnFg PartitionCount: 1       ReplicationFactor: 1   C
onfigs:
        Topic: food-orders      Partition: 0    Leader: 0       Replicas: 0     Isr: 0  Elr: N/A       L
astKnownElr: N/A

C:\kafka>
```

Figure 1. Kafka topic configuration verified in Windows CMD.

```
C:\kafka>bin\windows\kafka-console-consumer.bat --bootstrap-server localhost:9092 --topic food-orders --
from-beginning --max-messages 5
{"order_id": "ORD00001", "event_time": "2026-01-01T12:00:01", "restaurant": "Spice Hub", "order_amount":
 522.84, "delivery_minutes": 62}
{"order_id": "ORD00002", "event_time": "2026-01-01T12:00:02", "restaurant": "Pizza Corner", "order_amoun
t": 293.27, "delivery_minutes": 23}
{"order_id": "ORD00003", "event_time": "2026-01-01T12:00:03", "restaurant": "Green Bowl", "order_amount"
: 583.98, "delivery_minutes": 20}
{"order_id": "ORD00004", "event_time": "2026-01-01T12:00:04", "restaurant": "Spice Hub", "order_amount":
 492.01, "delivery_minutes": 16}
{"order_id": "ORD00005", "event_time": "2026-01-01T12:00:05", "restaurant": "Pizza Corner", "order_amoun
t": 179.03, "delivery_minutes": 47}
Processed a total of 5 messages

C:\kafka>
```

Figure 2. Consumer retrieving JSON food-order messages from Kafka.

6. Windowed Stream Processing (Layer 3)

The Python KafkaConsumer reads events, checks for existing order IDs, and calculates a rolling mean using up to the last 30 order amounts. The processor restores its window from the database when restarted, writes an order and updates aggregate metrics in one transaction, and commits the Kafka offset after successful database commit.

```
window = deque(previous_amounts, maxlen=30)
candidate = list(window) + [amount]
rolling = sum(candidate[-30:]) / len(candidate[-30:])
# Insert order; update metrics; commit MySQL; then commit Kafka offset
```

The rolling average smooths ordinary fluctuations while large order spikes remain visible on the dashboard chart.

7. Machine-Learning Model (Layer 4)

7.1 Offline training

Isolation Forest is an unsupervised anomaly detector. The training script fits the model on two numerical features: order_amount and delivery_minutes. The project uses 150 estimators, contamination=0.025, and random_state=42. The trained model is saved as anomaly_model.pkl.

```
X = df[['order_amount', 'delivery_minutes']]
model = IsolationForest(n_estimators=150, contamination=0.025,
                        random_state=42)
model.fit(X)
joblib.dump(model, ROOT/'anomaly_model.pkl')
```

7.2 Live scoring

The stream processor loads the saved model once. For each order, predict returns -1 for an outlier and 1 for a normal observation; the project stores is_anomaly=1 when the prediction is -1. These are statistical flags, not verified fraudulent transactions.

```
features = pd.DataFrame([[amount, minutes]],
    columns=['order_amount', 'delivery_minutes'])
anomaly = int(model.predict(features)[0] == -1)
```

```
C:\kafka>cd /d "E:\DSA MINI PROJECT\Kafka_Food_Delivery_Mini_Project\KafkaFoodDeliveryMiniProject"

E:\DSA MINI PROJECT\Kafka_Food_Delivery_Mini_Project\KafkaFoodDeliveryMiniProject>py -3.11 -c "from data
base import connect; c=connect(); cur=c.cursor(); cur.execute('SELECT order_id,restaurant,order_amount,d
elivery_minutes FROM orders WHERE is_anomaly=1 ORDER BY id DESC LIMIT 10'); print('DETECTED ANOMALIES');
 print('-'*75); [print('Order:',r[0],'| Restaurant:',r[1],'| Amount:',r[2],'| Delivery:',r[3],'minutes')
 for r in cur.fetchall()]; cur.close(); c.close()"
DETECTED ANOMALIES
---------------------------------------------------------------------------
Order: ORD01500 | Restaurant: Biryani Express | Amount: 4608.86 | Delivery: 41 minutes
Order: ORD01447 | Restaurant: Spice Hub | Amount: 120.11 | Delivery: 65 minutes
Order: ORD01425 | Restaurant: Pizza Corner | Amount: 3979.61 | Delivery: 45 minutes
Order: ORD01350 | Restaurant: Dosa House | Amount: 3061.42 | Delivery: 52 minutes
Order: ORD01275 | Restaurant: Spice Hub | Amount: 4568.09 | Delivery: 20 minutes
Order: ORD01214 | Restaurant: Green Bowl | Amount: 680.45 | Delivery: 65 minutes
Order: ORD01200 | Restaurant: Spice Hub | Amount: 3475.13 | Delivery: 18 minutes
Order: ORD01181 | Restaurant: Biryani Express | Amount: 132.3 | Delivery: 65 minutes
Order: ORD01132 | Restaurant: Biryani Express | Amount: 120.92 | Delivery: 62 minutes
Order: ORD01125 | Restaurant: Dosa House | Amount: 2958.15 | Delivery: 62 minutes

E:\DSA MINI PROJECT\Kafka_Food_Delivery_Mini_Project\KafkaFoodDeliveryMiniProject>
```

Figure 3. Stored anomaly results, including high-value and unusual-duration orders.

8. MySQL Serving Layer (Layer 5)

The orders table stores event details, the rolling average and anomaly flag. The live_metrics table stores a single-row snapshot of order count, total revenue, peak order, anomaly count and rolling average. This allows the dashboard to read summary values efficiently.

```
orders(order_id UNIQUE, event_time, restaurant, order_amount,
       delivery_minutes, rolling_avg, is_anomaly)
live_metrics(metric_id PRIMARY KEY, total_orders, total_revenue,
             peak_order, anomaly_count, rolling_avg)
```

```
E:\DSA MINI PROJECT\Kafka_Food_Delivery_Mini_Project\KafkaFoodDeliveryMiniProject>py -3.11 -c "from data
base import connect; c=connect(); cur=c.cursor(); cur.execute('SELECT COUNT(*),ROUND(SUM(order_amount),2
),ROUND(AVG(order_amount),2),MAX(order_amount),SUM(is_anomaly) FROM orders'); r=cur.fetchone(); print('F
OOD DELIVERY - MYSQL DATABASE SUMMARY'); print('-'*55); print('Total Orders:',r[0]); print('Total Revenu
e: INR',r[1]); print('Average Order: INR',r[2]); print('Peak Order: INR',r[3]); print('Detected Anomalie
s:',r[4]); cur.close(); c.close()"
FOOD DELIVERY - MYSQL DATABASE SUMMARY
-------------------------------------------------------
Total Orders: 1500
Total Revenue: INR 732443.31
Average Order: INR 488.3
Peak Order: INR 4635.18
Detected Anomalies: 38

E:\DSA MINI PROJECT\Kafka_Food_Delivery_Mini_Project\KafkaFoodDeliveryMiniProject>
```

Figure 4. Independent MySQL query validating final metrics.

9. Live Dashboard (Layer 6)

Streamlit displays five KPI cards, a line chart comparing order amount and rolling average, a list of flagged anomalies, and a recent-orders table. The app opens a fresh MySQL connection when its view refreshes; the @st.fragment(run_every="2s") decorator causes automatic updates. A stable dashboard was verified at localhost:8501.

```
@st.fragment(run_every='2s')
def live_view():
    con = connect()
    # Read live_metrics and recent orders; render cards and chart
live_view()
```

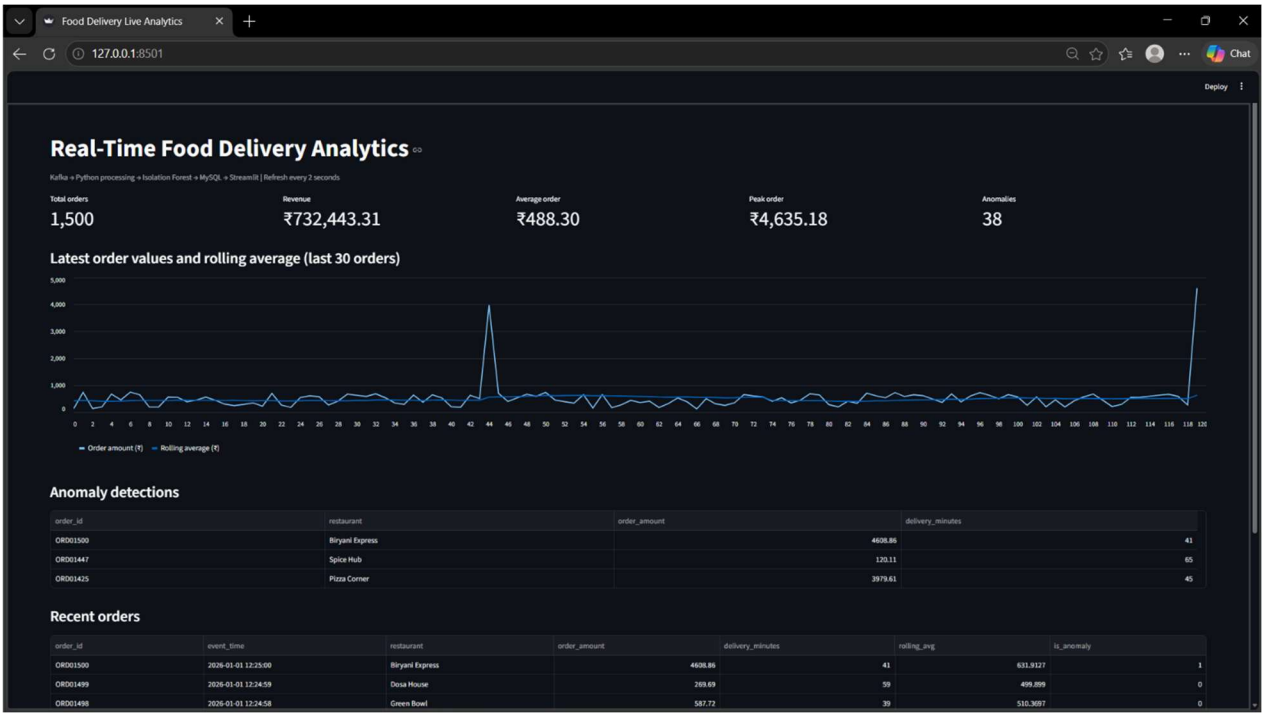


Figure 5. Working dashboard showing totals, rolling trend, anomalies and recent orders.

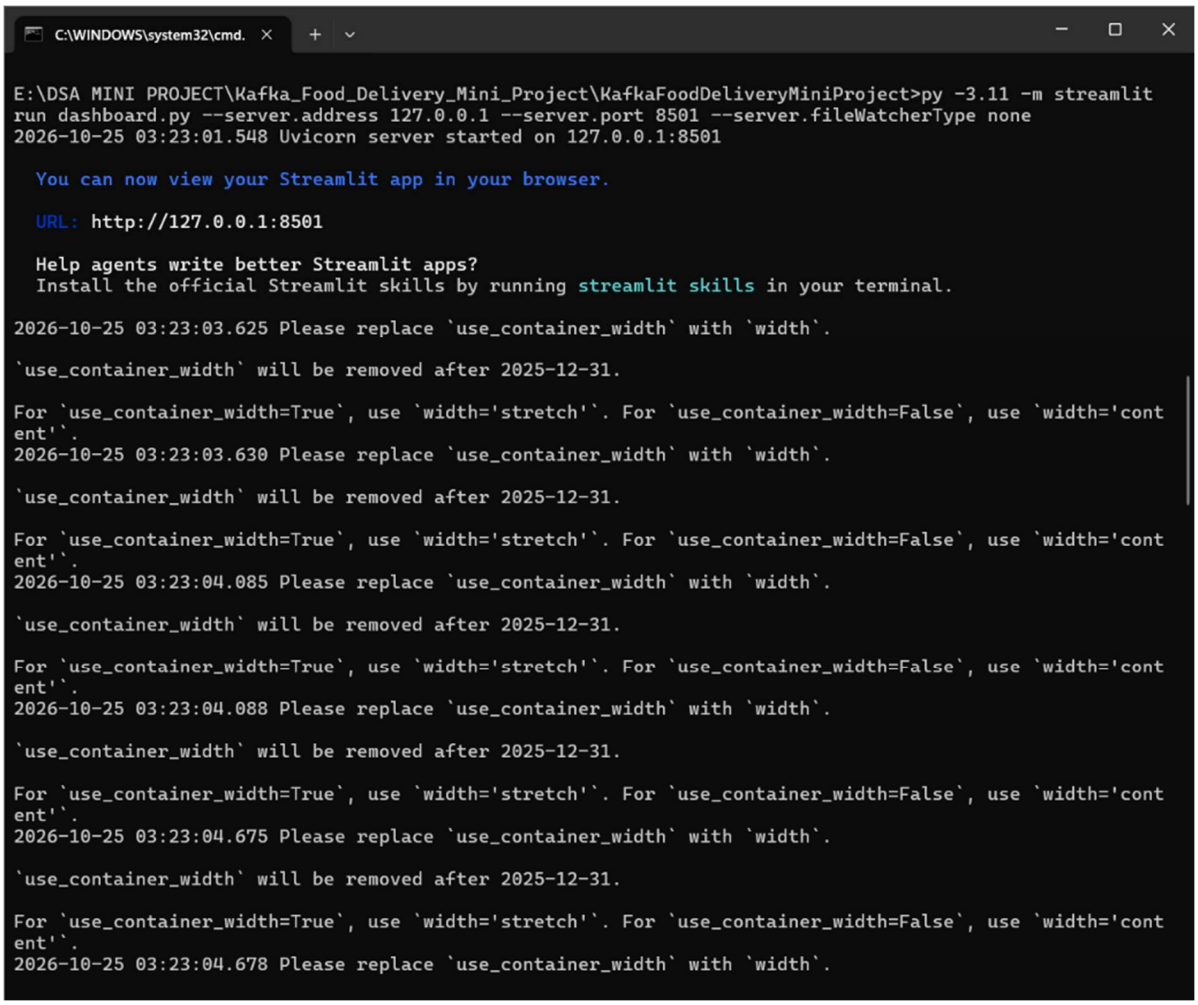


Figure 6. Streamlit server startup at localhost:8501.

10. Execution and Demonstration Procedure

Use separate terminals: (1) start ZooKeeper and the Kafka broker; (2) ensure MySQL is running; (3) run the Python processor; (4) run the Streamlit dashboard; (5) run demo_producer.py to publish 20 unique new orders. The producer sends one event every two seconds, and the dashboard refreshes about every two seconds. Do not rerun the fixed-ID demo to expect another increase: duplicate IDs are skipped by the processor.

```
py -3.11 processor.py
py -3.11 -m streamlit run dashboard.py --server.address 127.0.0.1 --server.port 8501 --
server.fileWatcherType none
py -3.11 demo_producer.py
```

A second live demonstration published 20 new unique orders (ORD01501–ORD01520), one every two seconds, while the Python processor and Streamlit dashboard were running. The processor scored and stored all 20 orders, and the dashboard increased from 1,500 to 1,520 orders. Before/after dashboard screenshots, producer output, processor logs and independent MySQL queries document the completed demonstration. The screenshots show before and after states; they do not constitute a continuous screen recording.

11. Experimental Results

Observed metric	Verified value
Orders processed	1,520
Total order value	₹7,48,233.31
Average order value	₹492.26
Peak order value	₹4,700.00
Flagged anomalies	40
Flagged share of orders	2.63%
Dashboard refresh interval	2 seconds

MySQL and the Streamlit dashboard independently confirm 1,520 processed orders, ₹748,233.31 in revenue, ₹492.26 average order value, ₹4,700.00 peak order and 40 flagged anomalies. The 20-order demonstration added ₹15,790.00 in revenue and two flagged anomalies (ORD01508 and ORD01516, each ₹4,700 with 95-minute delivery). No labeled ground truth was supplied, so detection precision/recall cannot be established.

12. Limitations and Future Enhancements

- Synthetic data cannot establish production accuracy or performance.
- Single-broker, one-partition configuration is intended for local demonstration, not high availability.
- Anomaly flags should be validated against labeled data before operational use.
- Capture an optional continuous screen recording for presentation, add explicit anomaly markers on the chart, and consider alert notifications.
- Extend to multiple Kafka brokers, scalable consumers, and model evaluation on labeled data.

13. Conclusion

The mini-project implements all six layers described in the guide: CSV source, Kafka transport, Python windowed processing, offline-trained Isolation Forest, MySQL storage and an auto-refreshing Streamlit dashboard. A completed 20-order live demonstration verified end-to-end processing, persistent storage and two new anomaly flags, with order totals rising from 1,500 to 1,520. This is a reproducible local proof of concept rather than a production-scale system.

Appendix A. Project Structure Evidence

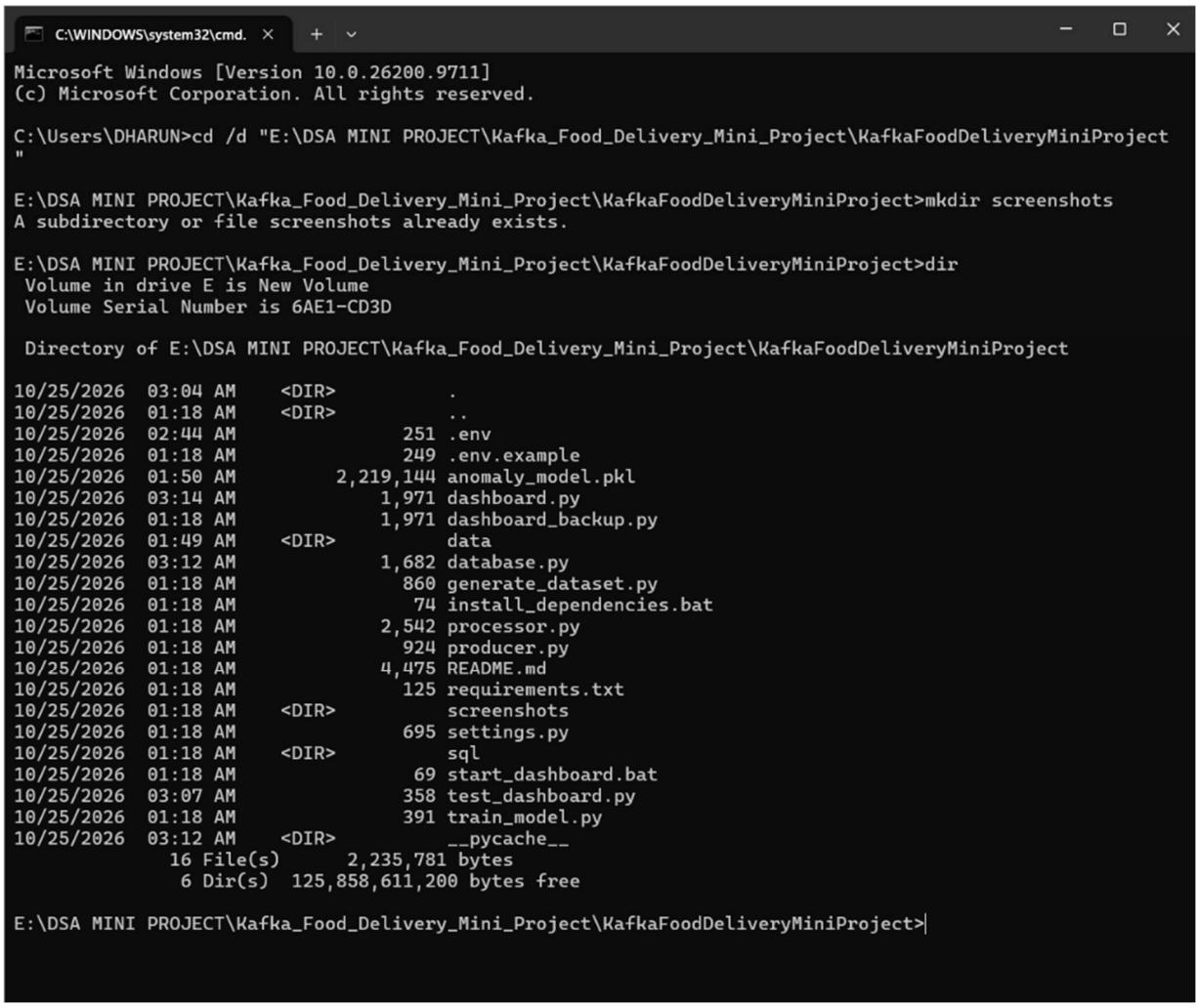


Figure 7. Project files used for generation, Kafka streaming, processing, ML, database and dashboard.

Appendix B. Guide Compliance Checklist

Guide item	Evidence / status
Six connected layers	Implemented; architecture and sections 4–9
ML trained and scored	train_model.py; processor.py; anomaly records
Auto-refreshing visualization	Dashboard code and screenshot
Database between processor and view	orders and live_metrics tables
Written report	This document
Live demo or recording	Completed 20-order live run; before/after screenshots, processor and producer logs, MySQL verification (no continuous recording)

Source basis: MiniProject_Guide(2).pdf, pages 3–9; provided project source files; initial execution screenshots; and 10 additional screenshots from the 25 October 2026 live demonstration.

Appendix C. Verified Live-Streaming Demonstration

Date: 25 October 2026. The original MySQL database held 1,500 orders. With ZooKeeper, Kafka, the Python consumer and Streamlit running, demo_producer.py published 20 new order events. All 20 were processed and saved. The two deliberately unusual orders, ORD01508 and ORD01516, were both flagged by the trained Isolation Forest model.

Measure	Before	After
Total orders	1,500	1,520
Total revenue	₹732,443.31	₹748,233.31
Average order	₹488.30	₹492.26
Peak order	₹4,635.18	₹4,700.00
Anomaly count	38	40

The new orders added ₹15,790.00 of revenue and 2 anomaly flags. The processor reported a final last-30-orders rolling average of ₹825.20. These are observed results, not estimates.

C.1 Baseline and final dashboard

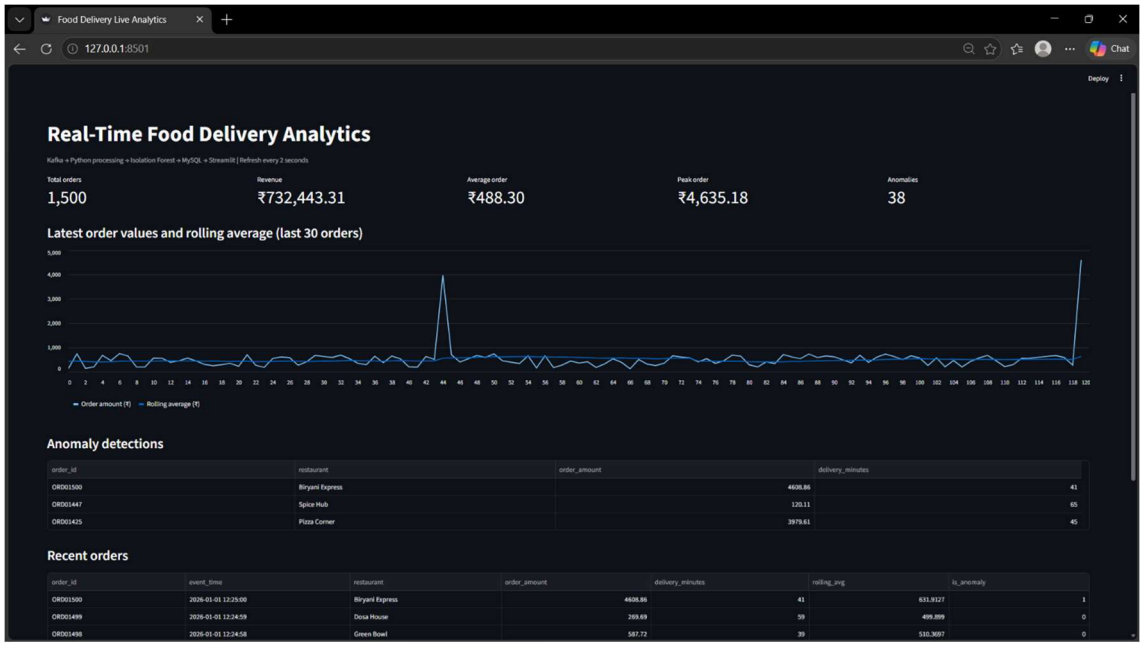


Figure 8. Before live streaming: 1,500 orders, ₹732,443.31 revenue, 38 anomalies.

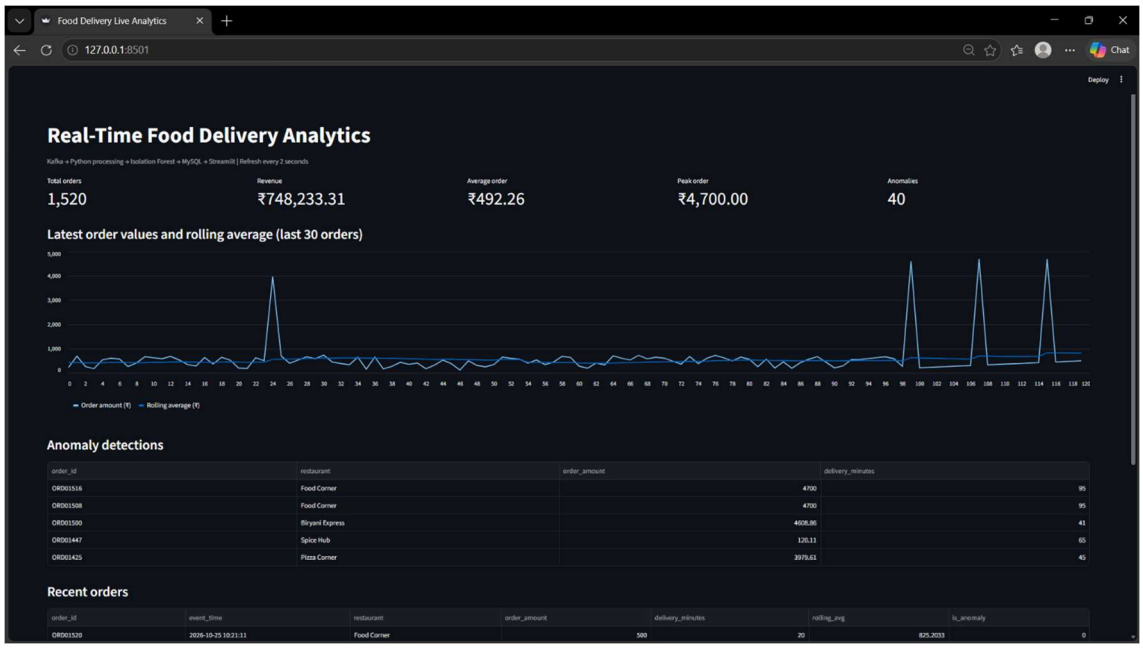


Figure 9. After streaming: 1,520 orders, ₹748,233.31 revenue, 40 anomalies; the two new flagged orders appear in the anomaly table.

C.2 Producer and processor evidence

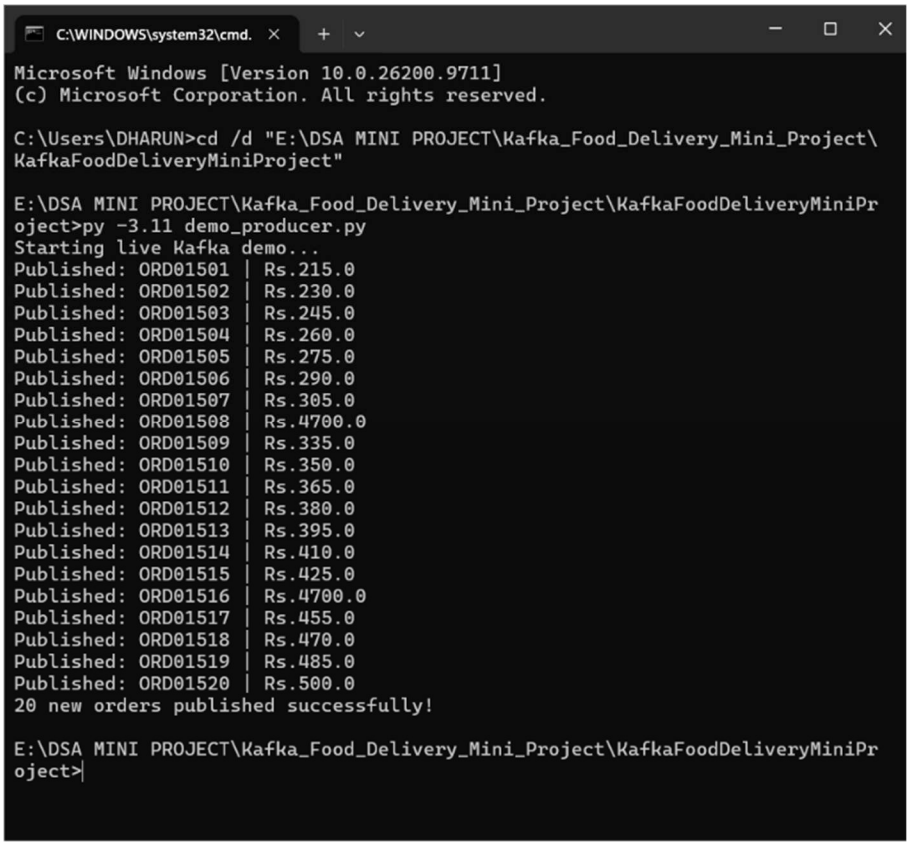


Figure 10. Producer confirms publication of all 20 new orders (ORD01501–ORD01520).

```
E:\DSA MINI PROJECT\Kafka_Food_Delivery_Mini_Project\KafkaFoodDeliveryMiniPr
oject>py -3.11 processor.py
E:\DSA MINI PROJECT\Kafka_Food_Delivery_Mini_Project\KafkaFoodDeliveryMiniPr
oject\processor.py:17: DeprecationWarning: value_deserializer does not imple
ment kafka.serializer.Deserializer
  consumer=KafkaConsumer(TOPIC,bootstrap_servers=BROKER,group_id='food-deliv
ery-processor-v1',auto_offset_reset='earliest',enable_auto_commit=False,valu
e_deserializer=lambda b:json.loads(b.decode('utf-8')))
Processor listening on food-orders
Processed ORD01501 amount=215.00 rolling=618.68 anomaly=0
Processed ORD01502 amount=230.00 rolling=610.19 anomaly=0
Processed ORD01503 amount=245.00 rolling=606.22 anomaly=0
Processed ORD01504 amount=260.00 rolling=592.35 anomaly=0
Processed ORD01505 amount=275.00 rolling=588.73 anomaly=0
Processed ORD01506 amount=290.00 rolling=578.39 anomaly=0
Processed ORD01507 amount=305.00 rolling=564.32 anomaly=0
Processed ORD01508 amount=4700.00 rolling=700.08 anomaly=1
Processed ORD01509 amount=335.00 rolling=694.60 anomaly=0
Processed ORD01510 amount=350.00 rolling=684.56 anomaly=0
Processed ORD01511 amount=365.00 rolling=677.78 anomaly=0
Processed ORD01512 amount=380.00 rolling=681.88 anomaly=0
Processed ORD01513 amount=395.00 rolling=676.04 anomaly=0
Processed ORD01514 amount=410.00 rolling=682.92 anomaly=0
Processed ORD01515 amount=425.00 rolling=681.70 anomaly=0
Processed ORD01516 amount=4700.00 rolling=831.94 anomaly=1
Processed ORD01517 amount=455.00 rolling=833.02 anomaly=0
Processed ORD01518 amount=470.00 rolling=829.77 anomaly=0
Processed ORD01519 amount=485.00 rolling=823.51 anomaly=0
Processed ORD01520 amount=500.00 rolling=825.20 anomaly=0
```

Figure 11. Consumer processes 20 new events and flags ORD01508 and ORD01516 (anomaly=1).

C.3 Independent database verification

```
E:\DSA MINI PROJECT\Kafka_Food_Delivery_Mini_Project\KafkaFoodDeliveryMiniPr
oject>py -3.11 -c "from database import connect; c=connect(); q=c.cursor();
q.execute('SELECT COUNT(*), ROUND(SUM(order_amount),2), SUM(is_anomaly) FROM
 orders'); print('Total orders, revenue, anomalies:',q.fetchone()); q.close(
); c.close()"
Total orders, revenue, anomalies: (1520, 748233.31, Decimal('40'))

E:\DSA MINI PROJECT\Kafka_Food_Delivery_Mini_Project\KafkaFoodDeliveryMiniPr
oject>
```

Figure 12. MySQL verification: 1,520 orders, ₹748,233.31 total revenue and 40 anomalies.

Additional captured evidence includes ZooKeeper startup, Kafka broker logs, topic description, the 1,500-order baseline MySQL query and the processor listening before publication. These corroborate service setup and baseline state; the figures above prioritize the key before/after evidence.